MONASH University
Information Technology

FIT3159
Computer Architecture

Data Representation

Dr Chetan Arora, FHEA
Chetan.Arora@monash.edu

Dr Charith Jayasekara, FHEA, SMIEEE, MACS CP, MIEAust
Charith.Jayasekara@monash.edu

Dr Carlo Kopp, SMIEEE, AFAIAA, FLSS, Peng
Carlo.Kopp@monash.edu

Faculty Information Technology,
Monash University
Australia

Why Study Data Representation?

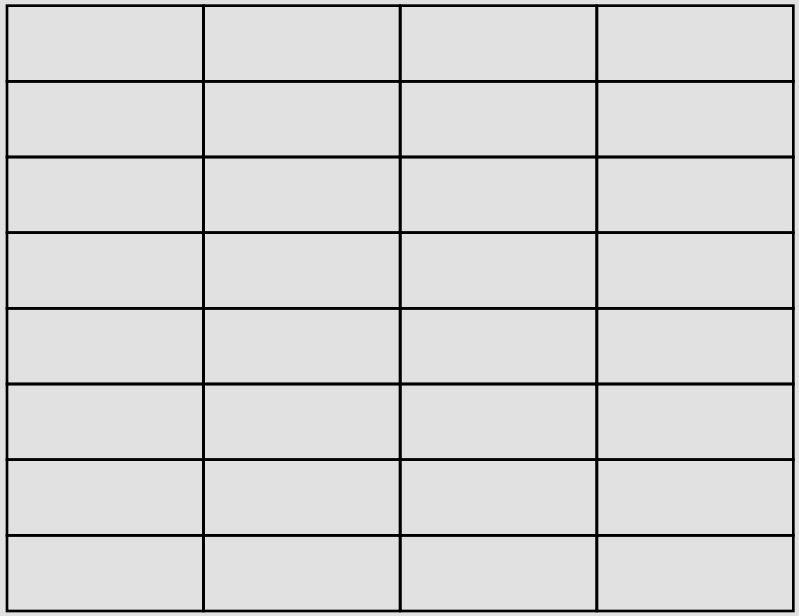
- **All data must be represented inside a computer in some form, so data representation is an unavoidable problem**
- **Designing computer hardware**
  - data representation is a critical constraint for designing registers, busses, arithmetic or execution units, and any other components through which the data must pass
- **Developing software**
  - making software portable across different machines is a common expectation and without understanding representation this is difficult to do
  - Designing datastructures for efficient storage (density) or access time (speed) requires a strong understanding of representation
  - Avoiding performance problems in I/O operations and cross platform networked applications requires a strong understanding of representation

MONASH University
Information Technology

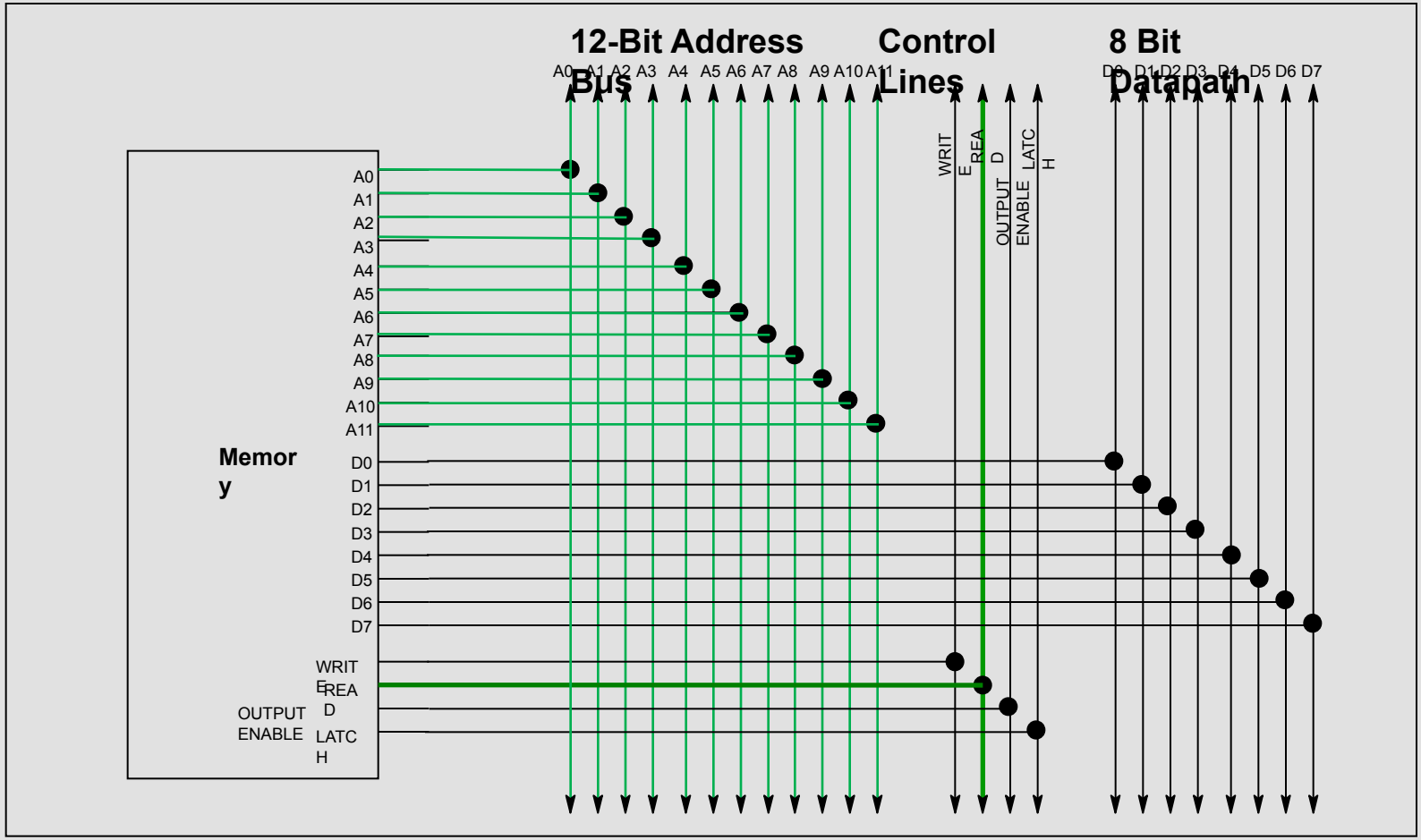
Memory Addressing

Addressing memory

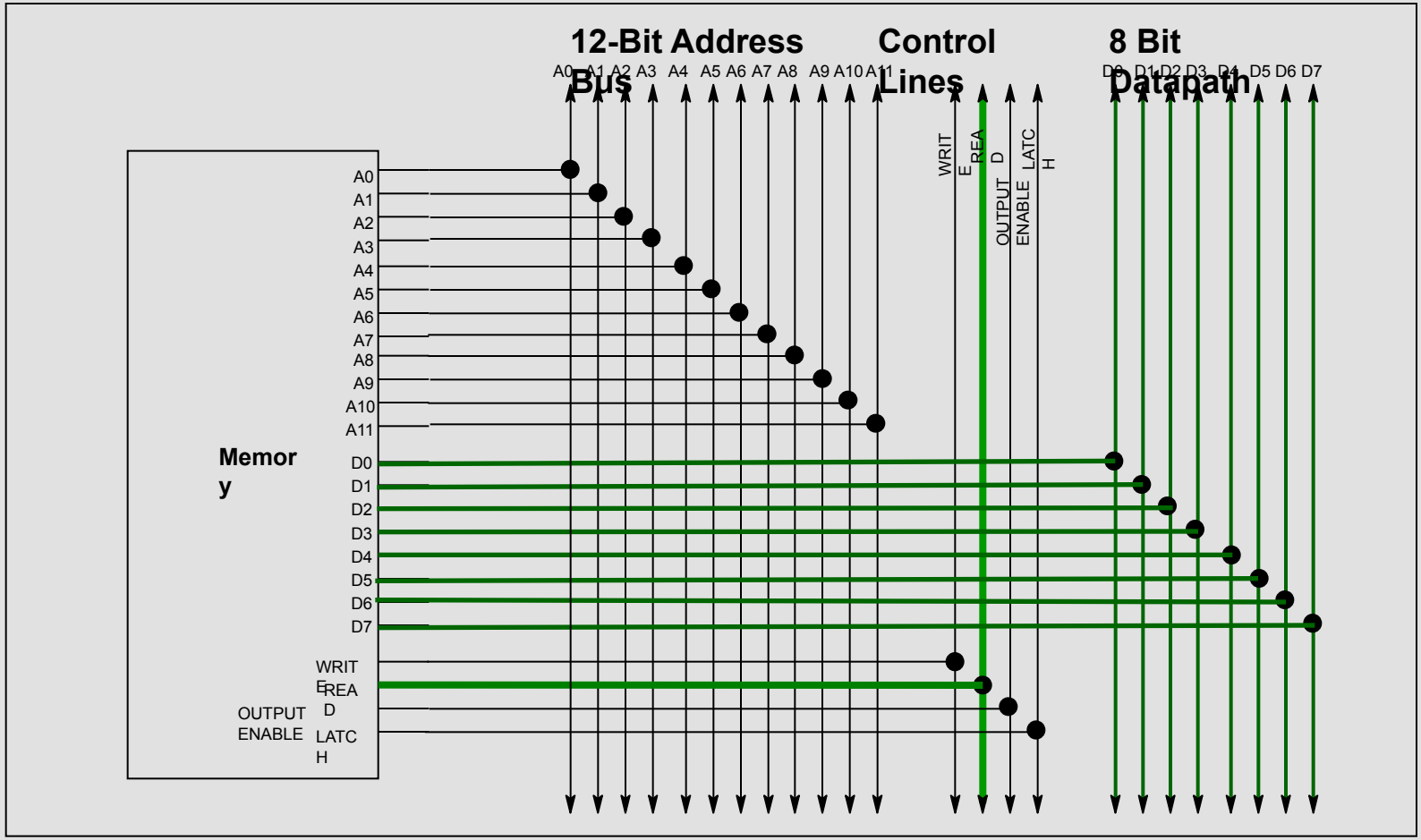
- **Every unit in memory has a unique address**



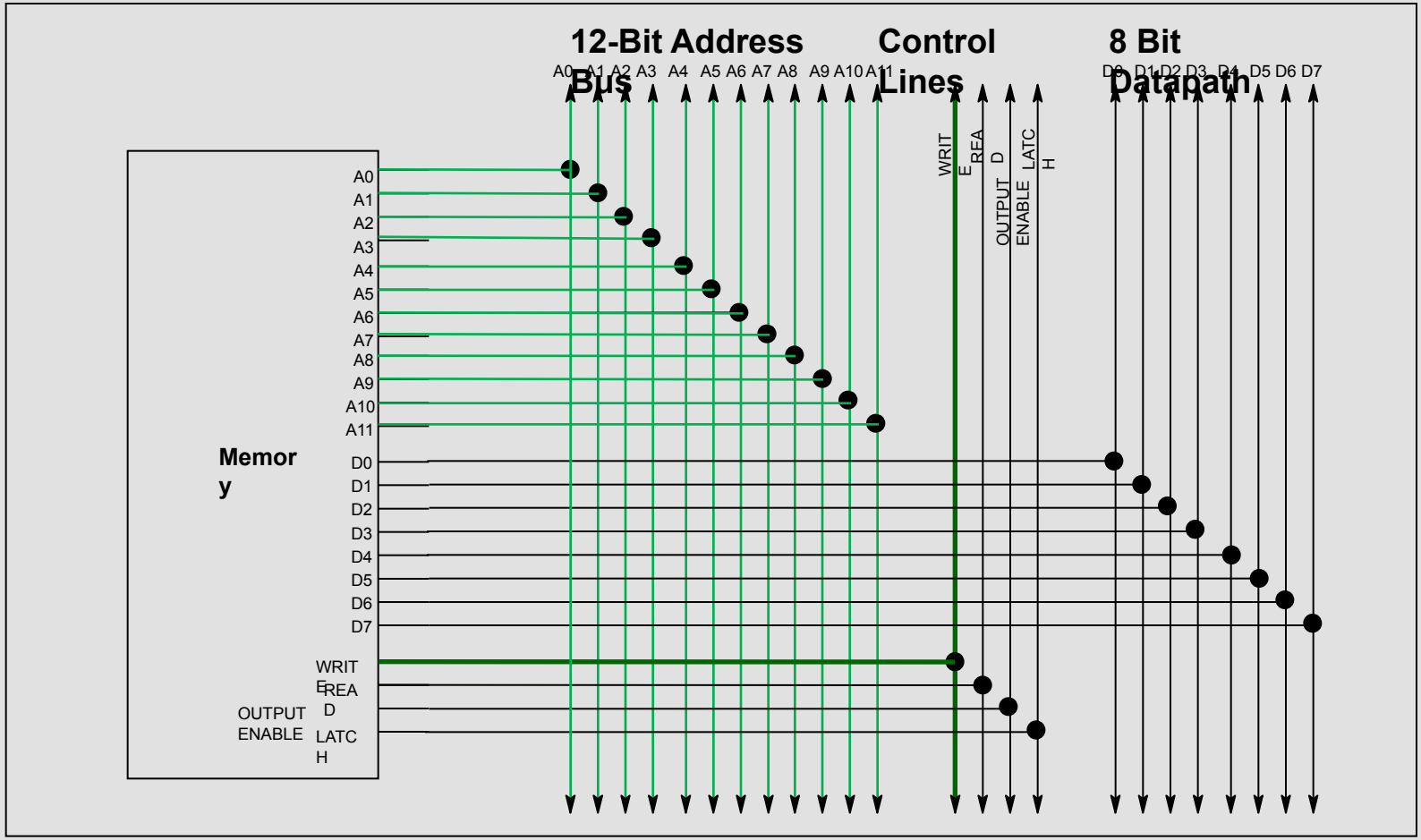
Reading: Assert Read Address and Control



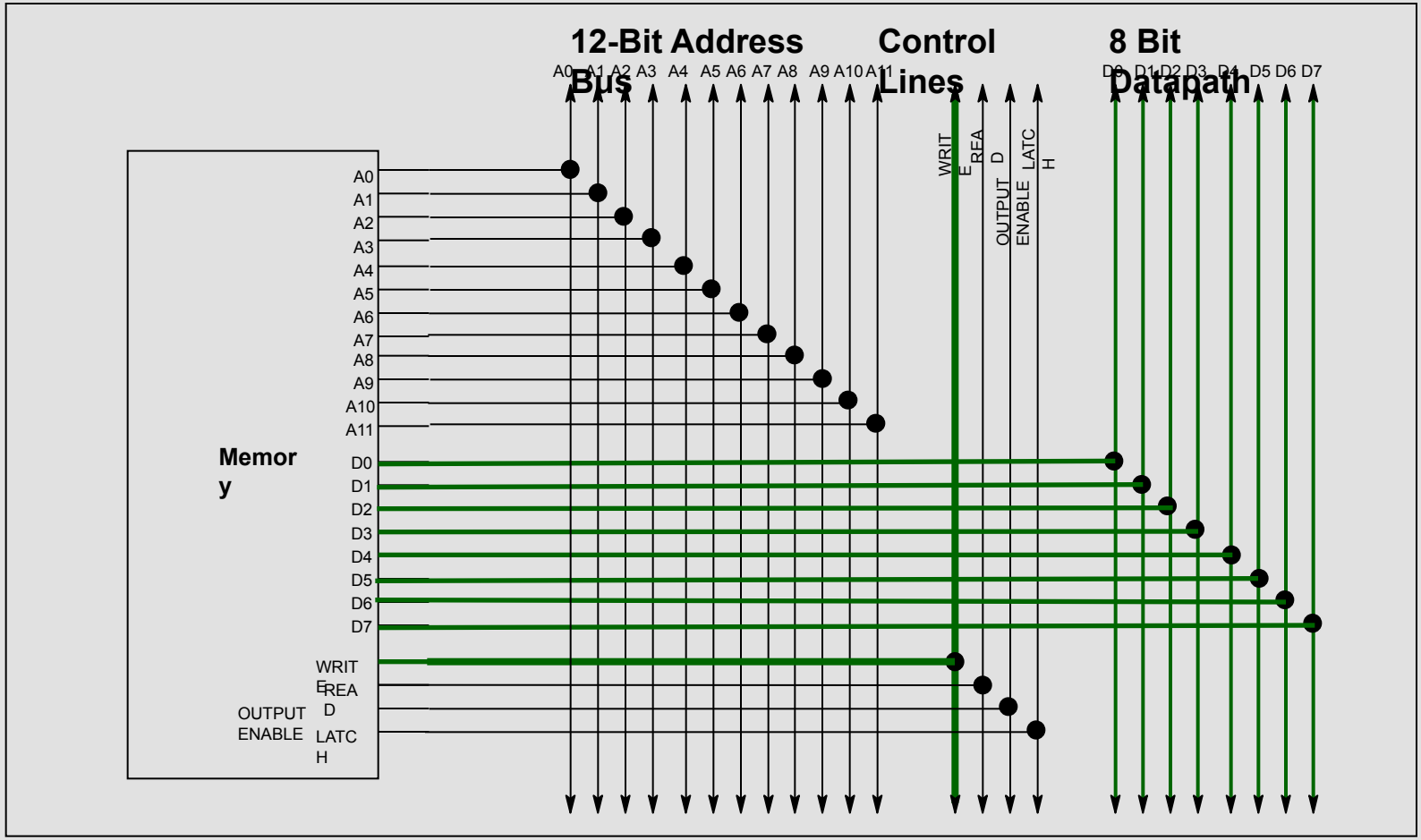
Reading From Memory: Obtain Read Data



Writing: Assert Write Address and Control

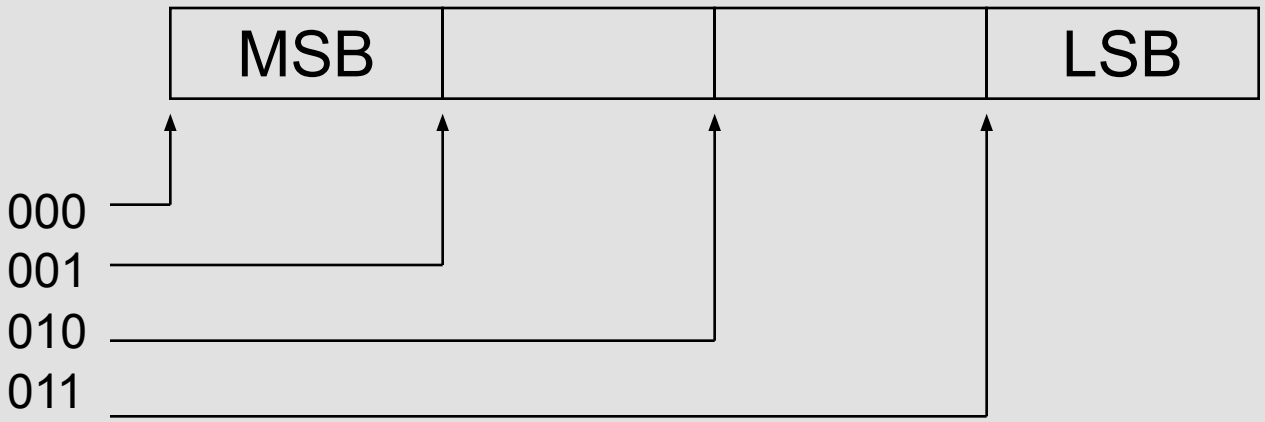


Writing: Assert Write Data



Byte addressing

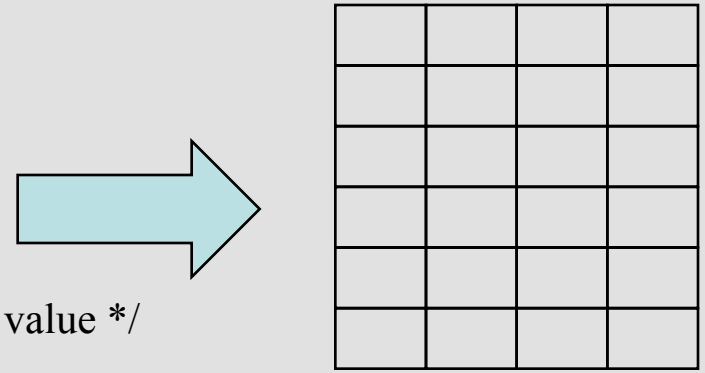
- **Machine address may allow finer resolution than word size**
  - E.g. 32 bit words may be addressed as 4 8 bit bytes



Data layout

- **Variables in high level language program need to be allocated to memory addresses**

```
void main{}
{
char value1;      /* ASCII character value */
short value2;     /* short integer value */
int value3; /* integer value */
long value4;      /* long integer value */
float value5;     /* floating point value */
double value6;    /* double precision floating point value */
fprintf( stdout, "hello world\n");
} /* end main */
```



Example of memory layout

```
void main{}
{
char value1;        /* ASCII character value */
short value2;       /* short integer value */
int value3;         /* integer value */
long value4;        /* long integer value */
float value5;       /* floating point value */
double value6;      /* double precision floating point
   value */
fprintf( stdout, "hello world\n");
} /* end main */
```

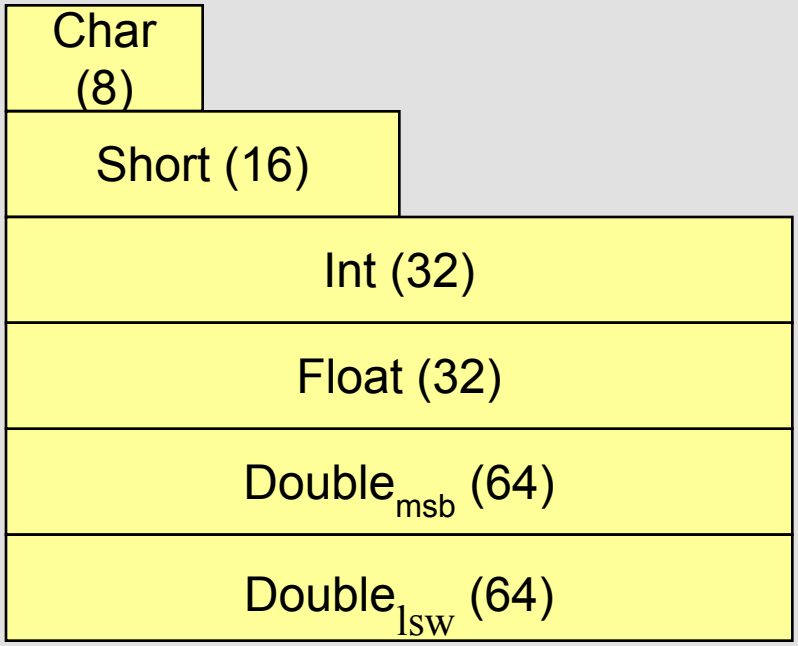
On 16 bit machine

<table>
<tr><td>Char (8)</td><td>Short (8)</td><td>Int (16)</td></tr>
<tr><td colspan="3">Long (32)</td></tr>
<tr><td colspan="3">Float (32)</td></tr>
<tr><td colspan="3">$Double_{msb}$ (64)</td></tr>
<tr><td colspan="3">$Double_{lsw}$ (64)</td></tr>
</table>

On 32 bit machine

<table>
<tr><td>Char (8)</td><td>Short (16)</td><td>Int (32)</td></tr>
<tr><td colspan="2">Int (32)</td><td>Float (32)</td></tr>
<tr><td colspan="2">Float (32)</td><td>Dbl (64)</td></tr>
<tr><td colspan="3">Double_{msb} (64)</td></tr>
<tr><td colspan="2">Double_{lsw} (64)</td><td></td></tr>
</table>

Without byte addressing?

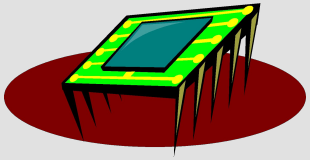


MONASH University
Information Technology

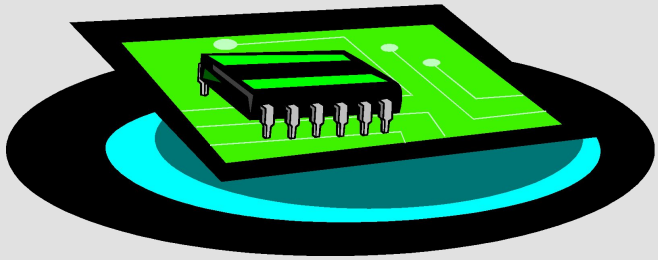
Data representations

Data Representation

- **Different data types require different amounts of storage.**
- **How much storage is specific to the architecture of the machine.**
- **A machine always has a “native word size” for a basic integer operand.**



Intel 4004
8 bit micro

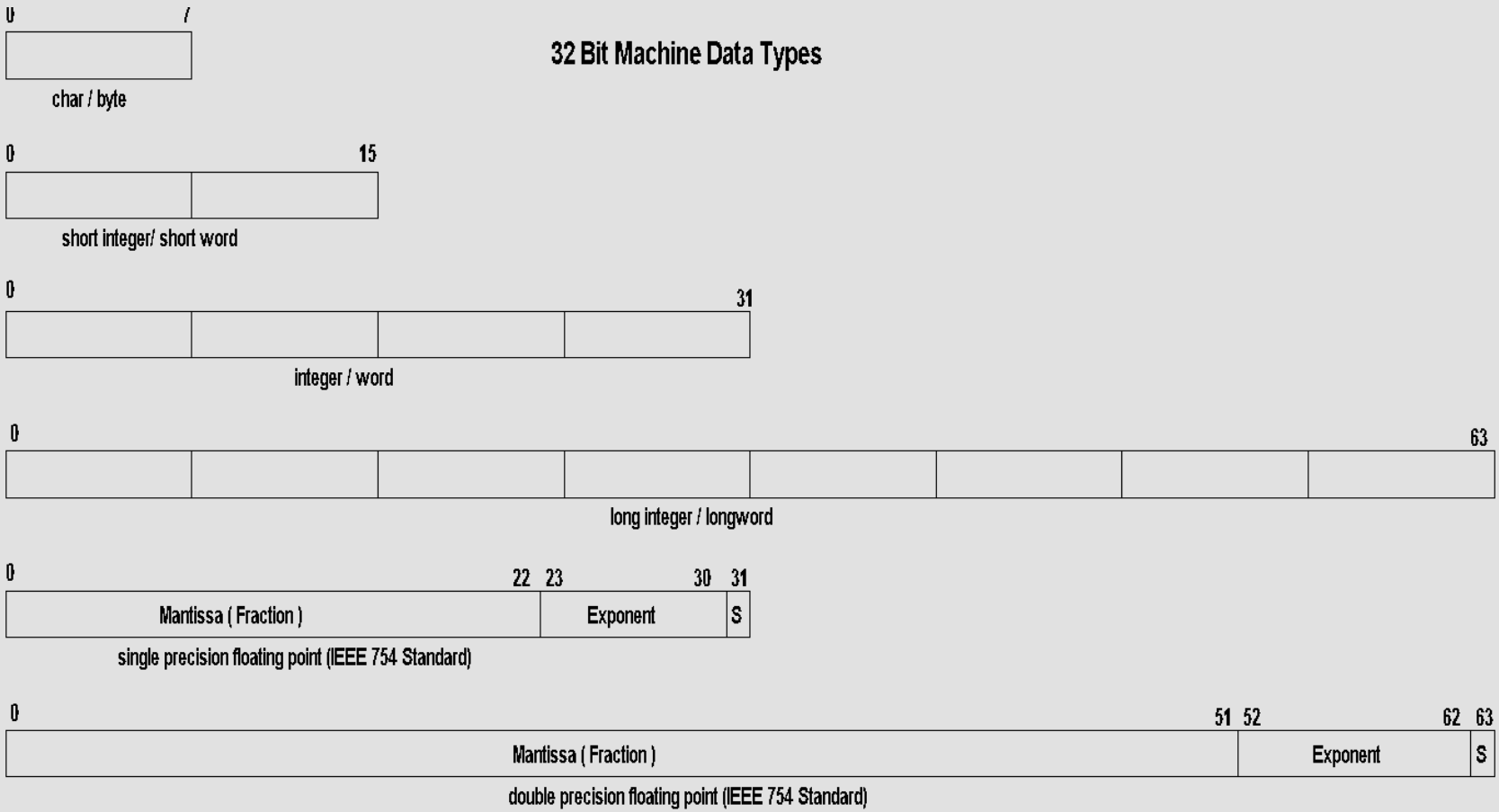


Intel Itanium
64 bit micro

Data Representation

- *"char"* = 1 byte = 8 bits = 8 D-latches = 1 ASCII character
- *"int"* = 1, 2, 4, 8 bytes = 8, 16, 32, 64 bits depending on machine.
- *"short"* = usually one half of "int" e.g. 1, 2, 4, 8 bytes ...
- *"long"* = usually two "int" i.e. 2, 4, 8, 16 bytes ...
- *"float"* = single precision floating point.
- *"double"* = double precision floating point.

Data Representation (32 Bit Machine)



Data Representation

- **Important for compiler design.**
- **Important for data structures in programming.**
- **Important for machine performance.**
  - What happens if word straddles a word boundary?
- **Important for machine address space size.**
- **The larger the number of bits, the more hardware is required.**
  - Affects bus widths, cache sizes, ALU design, …
- **Can vary widely between different architectures and generations of machine.**

Byte Ordering – Little vs Big Endian

- The order in which the bytes which form a multiple byte scalar value are stored in memory, is referred to as “byte ordering” or “endianness”.
- For instance, a 32-bit word comprises 4 bytes, starting from the most significant byte and ending with the least significant byte.
- The order of these bytes within the word depends upon the design of the CPU and memory subsystem in question.

Byte Ordering

- Assume the hex number 0x12345678 stored at address location 0x184.
- *Big Endian* has the MSB at the lowest address, *Little Endian* has the LSB at the lowest address.

BIG ENDIAN

184	12	MSB
185	34	
186	56	
187	78	LSB

LITTLE ENDIAN

184	78	LSB
185	56	
186	34	
187	12	MSB

Which is better?

- **Religious argument for some years**
- **Little Endian**
  - Address of word is same as least sig byte
  - Same address for short and long
    - > Difference is in the instruction that fetches the data
- **Doesn't make much difference**

Impact of Byte Ordering

- **Binary data files produced by Little Endian and Big Endian CPUs are not interchangeable.**
- **Therefore software which must manipulate such files must be capable of reordering the bytes in words or longwords. Examples:**
  - File serving protocols such as NFS incorporate mechanisms to reorder binary data (XDR protocol), to circumvent this.
  - Font files in X11 (Xwindows) must be recompiled to the proper byte order for the CPU architecture to be used.

MONASH University
Information Technology

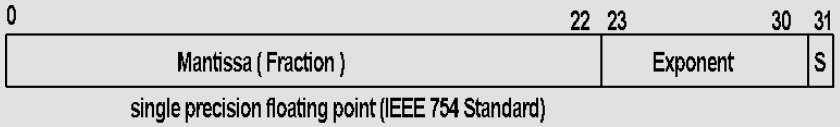
Floating Point Representation of Data

Floating Point Data

- Integer representations map directly into the bitwise inputs and outputs of functional blocks such as adders and registers.Therefore manipulating integers is very simple, since there is a direct correspondence between the integer value and the bits in the operand.
- Floating point data is structured differently, since it must represent real numbers.
- Numerous floating point representations exist, the IEEE format is frequently preferred in modern systems.

Floating Point Data

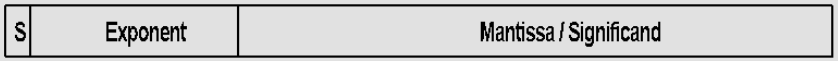
IEEE Floating Point Data Types



single precision floating point (IEEE 754 Standard)

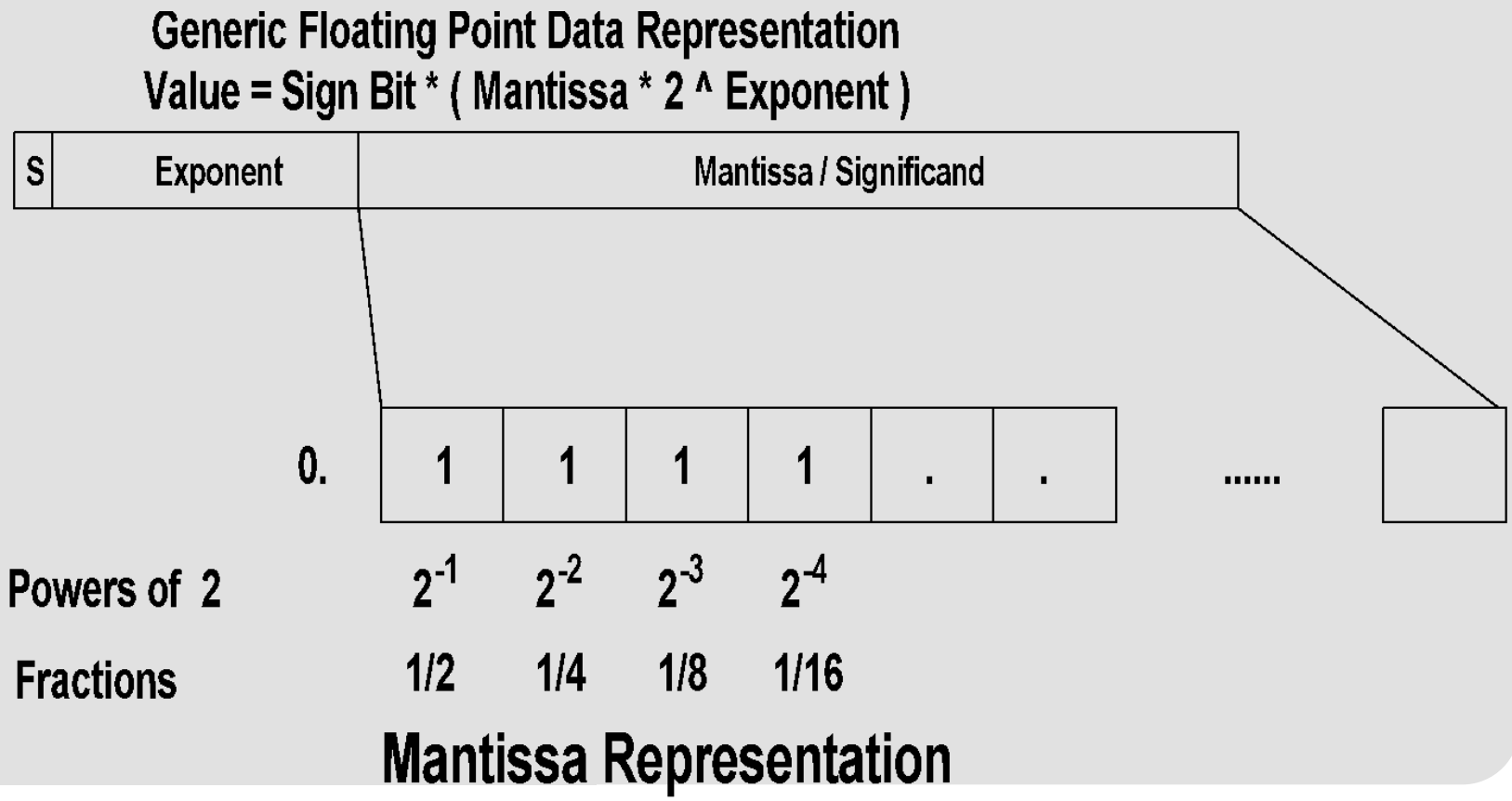
double precision floating point (IEEE 754 Standard)

IEEE Floating Point Data Types



Value = Sign Bit * (Mantissa * 2 ^ Exponent)

Mantissa


Generic Floating Point Data Representation
Value = Sign Bit * (Mantissa * 2 ^ Exponent)
S
Exponent
Mantissa / Significand
0.
1
1
1
1
.
.
.....
Powers of 2
2^{-1}
2^{-2}
2^{-3}
2^{-4}
Fractions
1/2
1/4
1/8
1/16
Mantissa Representation

The Mantissa

- The length of the mantissa determines the precision of a floating point number.
- The mantissa is a value between 0.10000... and 0.11111... , i.e. between ½ and 1, in a `normalised' floating point number.
- The first bit, representing ½, is always '1' in a normalised mantissa, therefore in some floating point schemes it is not stored.

The Exponent

- The length of the exponent determines the range (size) of a floating point number.
- If the exponent > 0, then the number > 1 (-1).
- If the exponent < 0, then the number < ½ (- ½).
- If the exponent == 0, then the number is between ½ and 1 (- ½ and -1).
- The exponent is usually in two's complement form.

An example

- $.375 = .25 + .125 \times 2^0$

  $= .01 + .001$

  $= .011$

  $= .11 \times 2^{-1}$

1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	• •	1	1	1	1	1	1	1	1

mantissa

exponent

Note, this is NOT IEEE format

Arithmetic on floating point numbers

- **Addition**
  - Denormalise mantissas
    - > Make exponents the same
  - Add
  - Normalise
- **Multiplication**
  - Multiply mantissas
  - Add exponents
  - Normalise
- **Can be implemented**
  - Using software or microcode with conventional arithmetic units
  - Special purpose hardware

Addition

- **.375 + .125 = .5**

1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	••	1	1	1	1	1	1	1	1

+

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	••	1	1	1	1	1	1	1	0

Note, this is NOT IEEE format

Addition

- .375 + .125 = .5

1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	••	1	1	1	1	1	1	1	1

+

0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	••	1	1	1	1	1	1	1	1

Note, this is NOT IEEE format

Addition

- .375 + .125 = .5

0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	••	0	0	0	0	0	0	0	0

+

0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	••	0	0	0	0	0	0	0	0

Note, this is NOT IEEE format

Addition

- .375 + .125 = .5

0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	••	0	0	0	0	0	0	0	0

+

0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	••	0	0	0	0	0	0	0	0

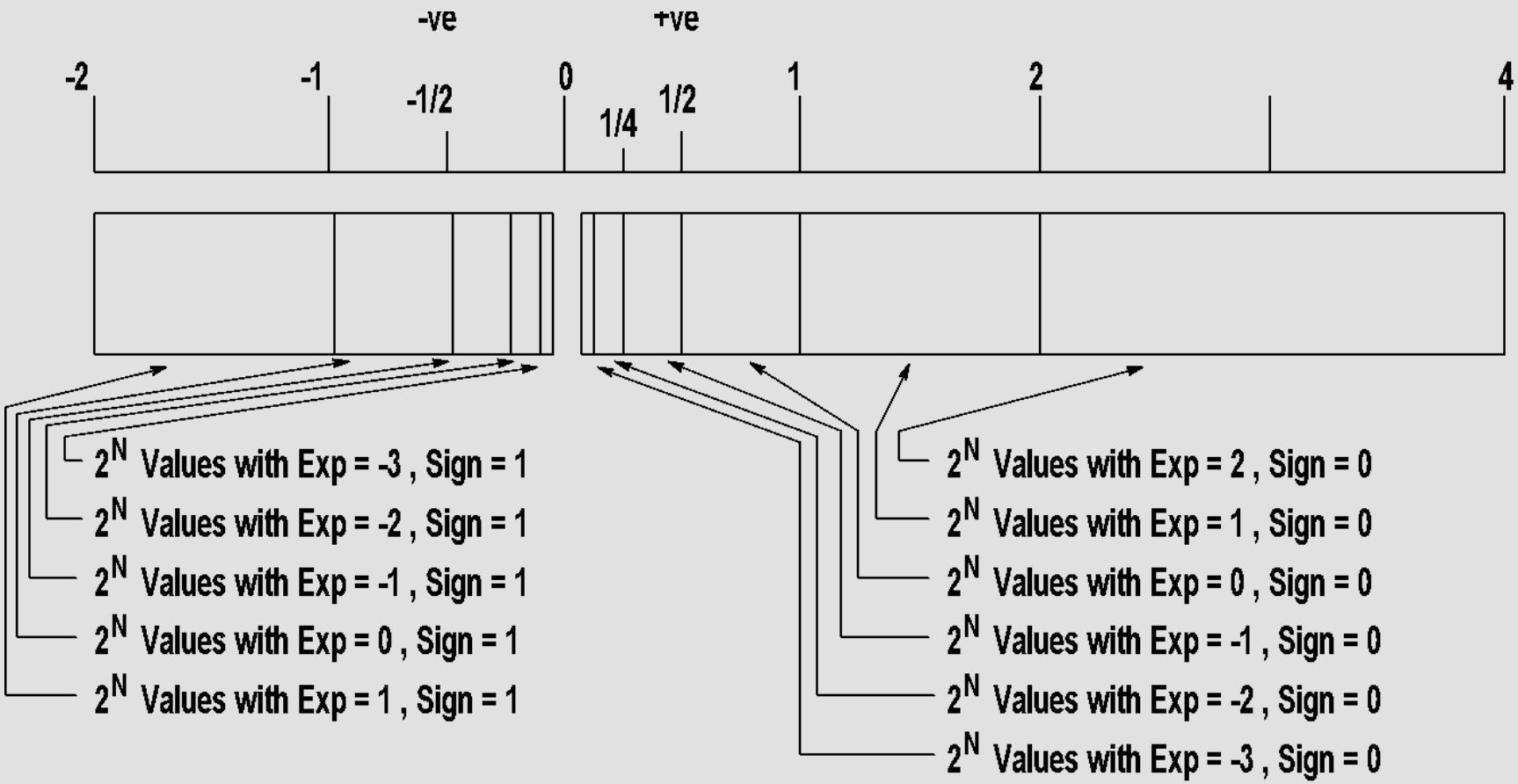
=

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	••	0	0	0	0	0	0	0	0

Note, this is NOT IEEE format

Real Numbers vs Floating Point

- There are an infinite number of real numbers between any two real numbers.
- When we represent real numbers in floating point form, we have a finite number of floating point values between any two numbers we can exactly represent as floating point numbers.
- Real numbers which cannot be exactly represented as floating point numbers can only be approximated, with some small error, given by the size of the mantissa.



Graphical Representation

Errors in Representation

- Example: 1/3 in a floating point form would be 0.333333333 which is inexact.
- The size of the error is (decimal):

  Error == 1/3 - 0.333333333
- The larger the number of bits in the mantissa, the smaller we can make this error.
- Error control in numeric programs is an important design consideration

Summary

- Representation of integers
- Little versus Big Endian
- Representation of floating point numbers
- Machine basic word sizes
- IEEE format
- Reals versus floats